

BÁO CÁO: SỬ DỤNG TRÍ TUỆ NHÂN TẠO (AI) CÓ TRÁCH NHIỆM TRONG HỌC THUẬT

Họ và tên: Trần Hoàng Minh Mã sinh viên: 25023494

I. Phân tích chính sách sử dụng AI trong môi trường Đại học

Tại các trường đại học khối kỹ thuật và công nghệ, chính sách sử dụng Generative AI đang dần được định hình theo hướng "**Thích ứng và Quản lý**" thay vì cấm đoán.

- **Điểm chính của chính sách:** AI được xem là một công cụ hỗ trợ (như máy tính bỏ túi trong toán học), sinh viên được phép sử dụng để tìm kiếm ý tưởng, giải thích khái niệm phức tạp (như các thuật toán toán học tối ưu, cấu trúc dữ liệu), và kiểm tra lỗi (debug) mã nguồn. Tuy nhiên, nghiêm cấm việc sao chép nguyên bản toàn bộ đoạn code hoặc văn bản do AI tạo ra để nộp làm bài của mình (đạo văn AI).
- **So sánh & Nhận định cá nhân:** So với các trường đại học truyền thống vốn yêu cầu cấm tuyệt đối AI trong các bài luận, chính sách của các trường công nghệ cởi mở hơn, phản ánh đúng thực tiễn ngành IT hiện nay (nơi GitHub Copilot hay ChatGPT là công cụ làm việc tiêu chuẩn). Dù vậy, thách thức cốt lõi vẫn là làm sao giảng viên đánh giá được chính xác năng lực cốt lõi của sinh viên (như tư duy thiết kế hệ thống, hiểu bản chất bộ nhớ trong Java) nếu sinh viên quá lạm dụng AI.

II. Thực hiện nhiệm vụ học tập với sự hỗ trợ của AI

1. Nhiệm vụ: Thiết kế kiến trúc phần mềm và sơ đồ lớp (Class Diagram) cơ bản cho đồ án môn học Lập trình hướng đối tượng (OOP) - Đề tài: Hệ thống Đấu giá trực tuyến (Auction System) áp dụng mô hình Client-Server.

2. Quá trình tương tác với AI:

- **Prompt đã sử dụng:** *"Đóng vai một kiến trúc sư phần mềm. Hãy giúp tôi lên dàn ý thiết kế kiến trúc Client-Server cho một Hệ thống Đấu giá (Auction System) viết bằng Java. Yêu cầu: Đề xuất cách áp dụng Design Pattern Singleton cho kết nối cơ sở dữ liệu và Factory Pattern cho việc phân quyền User/Admin. Chỉ cần mô tả kiến trúc và cấu trúc các lớp chính, không viết code chi tiết."*
- **Đầu ra của AI (Tóm tắt):** AI đã đề xuất một mô hình 3 lớp (Presentation, Business Logic, Data Access). Cụ thể:
 - Tạo lớp DatabaseConnection dùng Singleton để đảm bảo chỉ có 1 kết nối duy nhất từ Server đến DB.

- Tạo UserFactory để sinh ra các đối tượng StandardUser (người đấu giá) và AdminUser (quản trị viên).
- Gợi ý thêm việc sử dụng Socket/Multi-threading ở phía Server để xử lý cập nhật giá thầu (bids) theo thời gian thực (real-time).

Đóng vai một kiến trúc sư phần mềm. Hãy giúp tôi lên dàn ý thiết kế kiến trúc Client-Server cho một Hệ thống Đấu giá (Auction System) viết bằng Java. Yêu cầu: Đề xuất cách áp dụng Design Pattern Singleton cho kết nối cơ sở dữ liệu và Factory Pattern cho việc phân quyền User/Admin. Chỉ cần 

Chào bạn, với tư cách là một kiến trúc sư phần mềm, tôi đánh giá cao việc bạn định hình kiến trúc và các Design Pattern ngay từ giai đoạn đầu. Đối với một Hệ thống Đấu giá (Auction System), tính đồng bộ dữ liệu và khả năng quản lý kết nối là cực kỳ quan trọng.

Dưới đây là dàn ý thiết kế kiến trúc Client-Server và cách áp dụng các Design Pattern theo yêu cầu của bạn, hoàn toàn không đi sâu vào code chi tiết.

1. Tổng quan Kiến trúc Hệ thống (Client-Server Architecture)

Hệ thống sẽ được chia thành hai khối độc lập, giao tiếp với nhau qua mạng (thường là Socket TCP/IP hoặc RMI trong Java để đảm bảo tính thời gian thực khi đấu giá):

- **Server-side (Máy chủ):** Đóng vai trò là "bộ não" trung tâm. Chịu trách nhiệm quản lý toàn bộ Business Logic (xử lý phiên đấu giá, tính hợp lệ của giá bid), giao tiếp với Cơ sở dữ liệu (Database) và liên lạc liên tục với các Client đang kết nối.
- **Client-side (Máy khách):** Đóng vai trò là Presentation Layer (Giao diện người dùng). Chịu trách nhiệm hiển thị thông tin sản phẩm, nhận thao tác của người dùng (đặt giá, xem lịch sử), gửi yêu cầu lên Server và cập nhật giao diện khi nhận được luồng dữ liệu trả về từ Server.

2. Cấu trúc Lớp (Class Structure) và Ứng dụng Design Pattern

Chúng ta sẽ bóc tách hệ thống thành các layer chính với các class cốt lõi như sau:

A. Tầng Dữ liệu (Data Access Layer) - Áp dụng Singleton Pattern

Trong hệ thống đấu giá, Server sẽ phải xử lý hàng nghìn truy vấn cập nhật giá liên tục. Nếu mỗi thao tác đều tạo ra một kết nối mới đến Database, hệ thống sẽ nhanh chóng quá tải và sập.

- **Lớp `DatabaseConnectionManager` (Áp dụng Singleton):** * **Nhiệm vụ:** Quản lý kết nối cơ sở dữ liệu.
 - **Cơ chế Singleton:** Class này được thiết kế sao cho constructor là private. Nó giữ một biến static lưu trữ chính *instance* duy nhất của nó. Bất kỳ thành phần nào trên Server muốn truy xuất DB đều phải gọi một phương thức (ví dụ: `getInstance()`) để lấy kết nối từ một Pool duy nhất này. Điều này đảm bảo tài nguyên hệ thống được kiểm soát chặt chẽ.

B. Tầng Logic và Phân quyền (Business Logic Layer) - Áp dụng Factory Pattern

Hệ thống có các vai trò khác nhau (Người đấu giá, Quản trị viên) với những quyền hạn khác biệt (Admin có thể xóa phiên đấu giá, User chỉ có thể đặt giá).

- **Interface `User`:** Định nghĩa các phương thức chung mà mọi tài khoản đều có (ví dụ: `login()`, `viewProfile()`, `viewAuctions()`).
- **Các lớp Implementation:**
 - `StandardUser` (implements `User`): Bổ sung các quyền cụ thể như `placeBid()`, `viewBidHistory()`.

- o `AdminUser` (implements `User`): Bổ sung các quyền quản trị như `approveAuction()`, `banUser()`, `deleteItem()`.
- o **Lớp `UserFactory` (Áp dụng Factory Pattern):**
 - o **Nhiệm vụ:** Chịu trách nhiệm khởi tạo đối tượng người dùng.
 - o **Cơ chế:** Khi một client gửi yêu cầu đăng nhập thành công với một `role_type` nhất định từ Database, thay vì dùng từ khóa `new` rải rác khắp nơi, luồng xử lý sẽ gọi đến `UserFactory.createUser(role_type)`. Factory sẽ tự động kiểm tra điều kiện và trả về một instance của `StandardUser` hoặc `AdminUser` tương ứng. Điều này giúp ẩn đi logic khởi tạo phức tạp và dễ dàng mở rộng nếu sau này hệ thống có thêm vai trò mới (ví dụ: `VIPUser` hay `Auditor`).

C. Tầng Quản lý Đấu giá (Auction Core & Networking)

- o **Lớp `AuctionManager`:** Class cốt lõi trên Server quản lý các phiên đấu giá đang diễn ra. Nó giữ danh sách các mặt hàng, mức giá hiện tại, và đồng bộ hóa (synchronize) các luồng (threads) đặt giá để tránh tình trạng Race Condition (hai người cùng đặt giá tại một mili-giây).
- o **Lớp `ClientHandler`:** Mỗi khi một Client kết nối, Server tạo ra một luồng `ClientHandler` riêng để duy trì liên lạc liên tục, giúp push thông báo "Giá đã thay đổi" về giao diện theo thời gian thực.

3. Tổ chức Mô-đun (Module Breakdown)

Với kiến trúc Client-Server phân rã rõ ràng như trên, việc phát triển theo nhóm sẽ rất trơn tru và rạch ròi, tránh được tình trạng conflict code. Để tối ưu hóa, 4 khối lượng công việc chính này có thể được chia đều: Dững có thể tập trung thiết kế Client UI và event handling, Minh phụ trách xây dựng Server

3. Tổ chức Mô-đun (Module Breakdown)

Với kiến trúc Client-Server phân rã rõ ràng như trên, việc phát triển theo nhóm sẽ rất trơn tru và rạch ròi, tránh được tình trạng conflict code. Để tối ưu hóa, 4 khối lượng công việc chính này có thể được chia đều: Dững có thể tập trung thiết kế Client UI và event handling, Minh phụ trách xây dựng Server Socket & Auction Core (đồng bộ hóa đa luồng), Duy đảm nhận Data Access Layer cùng cấu trúc Database, và Bách có thể chịu trách nhiệm thiết kế, chuẩn hóa các Model và Design Patterns (Singleton, Factory) để cả nhóm cùng tích hợp.

Bạn dự định sẽ sử dụng Socket thuần (TCP/UDP) hay áp dụng RMI (Remote Method Invocation) cho việc xử lý giao tiếp mạng và cập nhật giá thời gian thực trong dự án này?

3. Đánh giá, chỉnh sửa và tích hợp:

- **Đánh giá:** Đầu ra của AI rất logic và chuẩn xác về mặt lý thuyết thiết kế mẫu (Design Patterns). Tuy nhiên, phần cập nhật real-time bằng Socket thô (raw sockets) mà AI gợi ý khá phức tạp cho phạm vi một đồ án OOP cơ bản của nhóm.

- **Chỉnh sửa & Tích hợp:** Tôi quyết định giữ lại cấu trúc Singleton và Factory vì nó đáp ứng đúng yêu cầu môn học. Tuy nhiên, tôi đã loại bỏ phần kiến trúc Socket phức tạp của AI, thay vào đó tự thiết kế lại luồng cập nhật giá thầu bằng phương pháp Polling đơn giản hơn để phù hợp với tiến độ và năng lực của 4 thành viên trong nhóm.
- **Trích dẫn sử dụng AI (Minh bạch):** *"Dàn ý kiến trúc phần mềm và cách áp dụng Design Patterns trong tài liệu đồ án này được khởi tạo với sự hỗ trợ tư vấn của mô hình ngôn ngữ lớn (LLM) vào tháng 5/2026. Các quyết định thiết kế cuối cùng và việc giới hạn phạm vi kỹ thuật đã được tác giả và nhóm dự án tự đánh giá và điều chỉnh để phù hợp với yêu cầu thực tế."*

III. Phân tích các vấn đề đạo đức

1. Ranh giới giữa hỗ trợ hợp lý và gian lận học thuật: Sử dụng AI để giải thích nguyên lý hoạt động của từ khóa `extends` trong Java, hoặc nhờ AI tìm lỗi (bug) trong một phương thức tính toán ma trận là **hỗ trợ hợp lý**. Ngược lại, yêu cầu AI viết toàn bộ module Quản lý phiên đấu giá từ đầu đến cuối và nộp đó như là code của mình chính là **gian lận**. Ranh giới nằm ở việc: "Ai là tác giả của tư duy giải quyết vấn đề?".

2. Vấn đề quyền sở hữu trí tuệ và trích dẫn: Các LLM học từ hàng tỷ dòng code mã nguồn mở trên GitHub. Khi AI tạo ra một đoạn mã tối ưu hóa cấu trúc dữ liệu, rất có thể nó đang "xào nấu" lại chất xám của một lập trình viên khác mà không hề ghi nguồn. Do đó, việc tự nhận đó là sáng chế của mình vi phạm nghiêm trọng đạo đức kỹ thuật. Cần phải có tuyên bố minh bạch về mức độ can thiệp của AI trong dự án.

3. Tác động đến quá trình học tập và phát triển kỹ năng: Lạm dụng AI gây ra hiệu ứng "ảo tưởng sức mạnh" (Dunning-Kruger). Sinh viên có thể tạo ra một chương trình chạy hoàn hảo nhưng lại không hiểu cơ chế cấp phát bộ nhớ hay vòng đời của đối tượng trong Java. Về lâu dài, điều này làm thui chột khả năng giải quyết vấn đề độc lập – kỹ năng sống còn của một kỹ sư IT.

IV. Bộ nguyên tắc cá nhân về sử dụng AI (6 Nguyên tắc)

1. **Nguyên tắc Khởi xướng:** Chỉ dùng AI khi đã tự mình suy nghĩ và phác thảo giải pháp/dàn ý ít nhất 15 phút mà vẫn bế tắc.
2. **Nguyên tắc "Tư duy, không phải Code":** Yêu cầu AI cung cấp thuật toán (Mã giả - Pseudocode) hoặc giải thích khái niệm, tuyệt đối không yêu cầu AI viết code hoàn chỉnh để copy-paste.
3. **Nguyên tắc Thăm định (Zero-Trust):** Luôn coi câu trả lời của AI là một bản nháp có thể sai. Phải đối chiếu chéo (cross-check) với tài liệu chính thống (như Oracle Docs cho Java) trước khi sử dụng.

4. **Nguyên tắc Tinh chỉnh:** Không bao giờ sử dụng đầu ra của AI nguyên bản 100%. Phải chỉnh sửa văn phong, logic và bối cảnh cho phù hợp với dự án cụ thể của nhóm.
5. **Nguyên tắc Minh bạch:** Luôn ghi chú (comment) trong mã nguồn hoặc báo cáo về những phần có sử dụng sự hỗ trợ của AI.
6. **Nguyên tắc Bảo mật:** Không nhập các dữ liệu cá nhân nhạy cảm, mã số sinh viên, hoặc mã nguồn nội bộ mang tính bảo mật cao vào các khung chat AI công cộng.